

Day 1 Summary

Path Parameters & Numeric Validations / Query Parameters & String Validations | Week 2, FastAPI & REST Principles

Concept A — Path Parameters and Numeric Validations

What it is: `Path()` lets you attach validation and metadata to a *path* parameter, the same way `Query()` does for query parameters. Common constraints: `gt` / `ge` (greater than / greater-or-equal) and `lt` / `le` (less than / less-or-equal) for numbers.

Why it matters: path parameters are always required (they're part of the URL itself), so `Path()` isn't for making something optional — it's purely for adding rules and better auto-generated docs, e.g. rejecting `item_id=0` before your function body ever runs.

Concept B — Query Parameters and String Validations

What it is: `Query()` adds string-specific constraints to a query parameter: `min_length`, `max_length`, and `pattern` (regex). It's declared as the parameter's default value.

Extra detail worth knowing: `alias=` lets a query parameter be named something in the URL that isn't a valid Python identifier (e.g. `item-query`), while your function still uses a clean Python variable name internally.

Practical Exercise — Combining Both

One endpoint exercising both: a validated path parameter and a validated query parameter together. Fill in the TODOs; don't copy a finished version.

```
from fastapi import FastAPI, Path, Query
from typing import Annotated

app = FastAPI()

# TODO 1: GET /items/{item_id}
#   item_id: Annotated[int, Path(gt=0, le=1000)]
#   -> must be a positive integer, max 1000
#   q: Annotated[str | None, Query(min_length=3, max_length=50)] = None
#   -> optional, but if given, must be 3-50 characters
#
#   return {"item_id": item_id, "q": q}

# TODO 2: test all four cases below in /docs and note what happens to each:
#   a) /items/5           (valid item_id, no q)
#   b) /items/0           (invalid - violates gt=0)
#   c) /items/5?q=hi      (invalid - q too short, violates min_length=3)
#   d) /items/5?q=hello   (valid - passes both checks)
```

Prediction Questions — Answer Before Testing in /docs

1. For case (b), `/items/0` — does your function body ever run, or does FastAPI reject the request before it gets there? What status code do you expect, and why does that matter for API design (should invalid input ever reach your business logic)?
2. For case (c), `q=hi` is 2 characters against a `min_length=3` rule — predict the exact error FastAPI returns before you run it. Does it name which field failed, and what the rule was?

Status: Day 1 In Progress

Next: fill in TODO 1, run the server, test all four cases from TODO 2 in `/docs`, then answer both prediction questions in your own words based on what you actually observed — same method as Week 1's timing predictions. Send the completed code back for a check before moving to Day 2 (already done — this closes the loop backward).